

计算机程序设计基础(2)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所
罗姆楼6-104

电话: 13901216180 / 62796193

2020. 2.

教学目标

- 本课主要是培养掌握面向对象的程序设计基本原理、概念、方法和编程解题的思路与典型方法，达到可分析和设计综合程序的能力。
- 掌握类的基本概念、类的封装、类的继承性和多态性、函数重载与运算符重载等C++语言的基本知识。
- 并进一步掌握VC++编程环境下的程序开发和调试方法。

教材和参考书

1. 教材

黄永峰, 孙甲松: 《C/C++程序设计教程》
清华大学出版社 2019年6月第1版

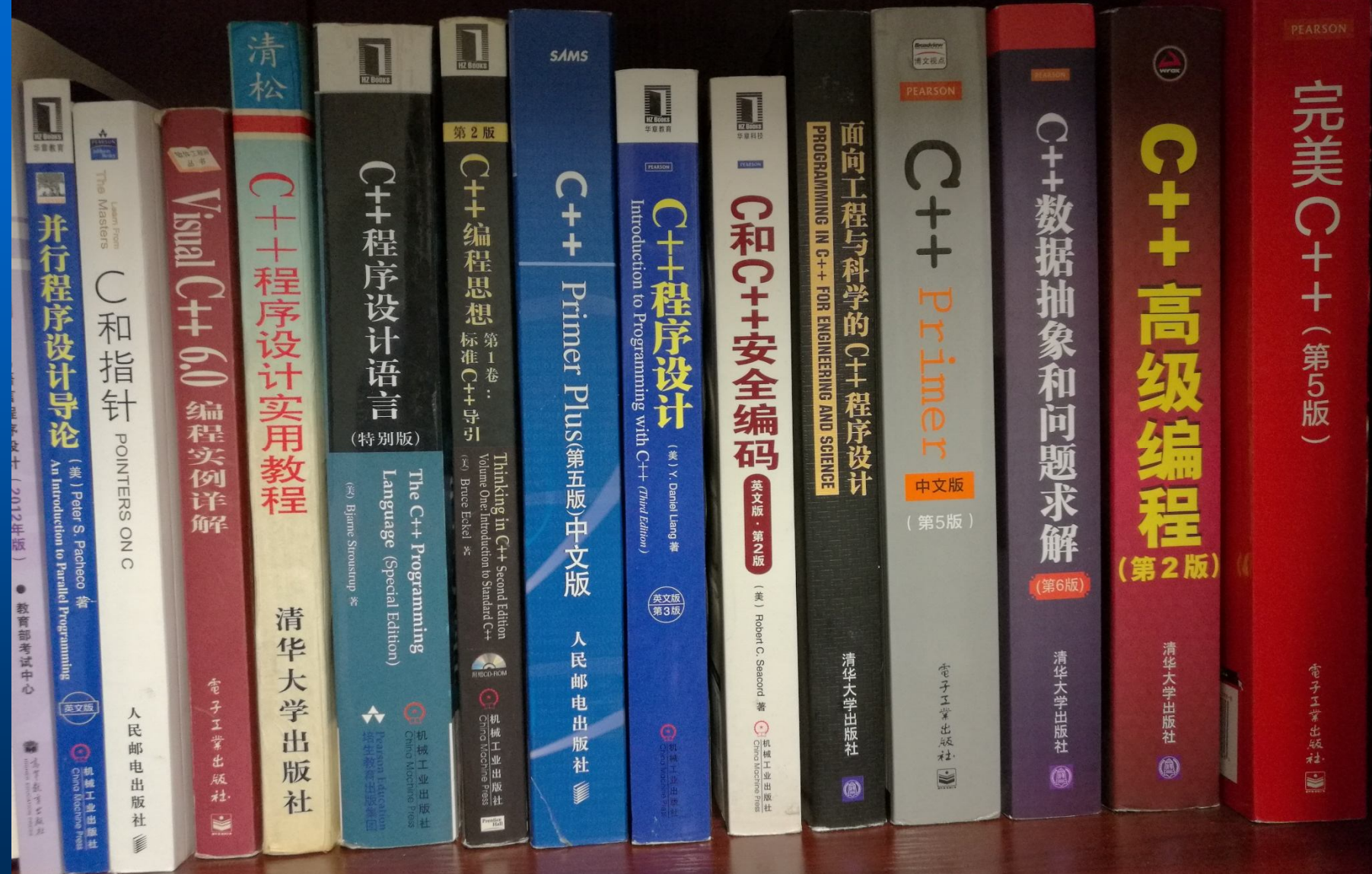
2. 主要参考书

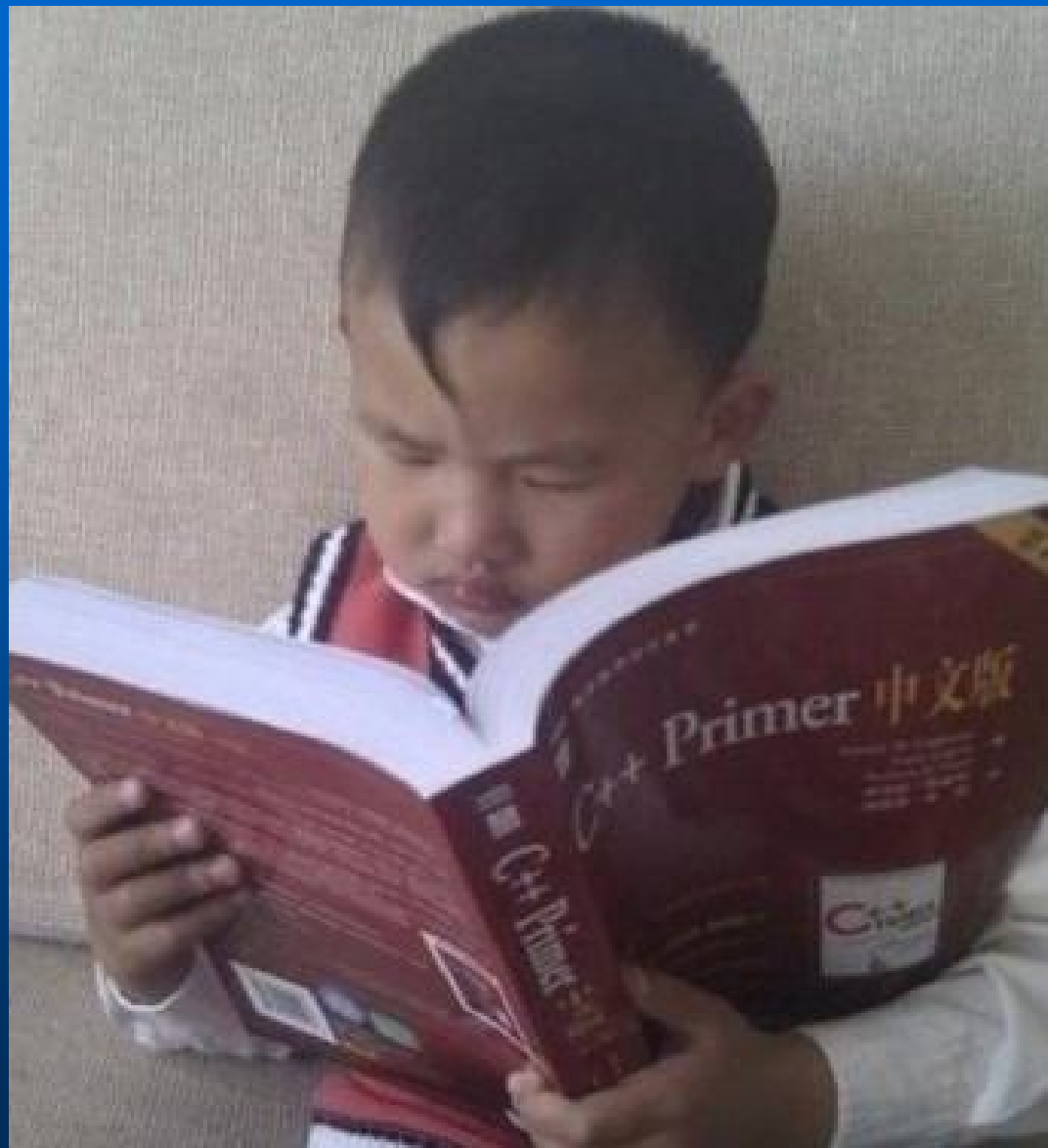
(1) 郑莉: 《C++语言程序设计(第4版)》 清华大学出版社

(2) Stephen Prata: 《C++ Primer Plus 中文版》 第5版,
人民邮电出版社

(3) Bruce Eckel: 《C++编程思想(Thinking in C++, 2nd/plus ed)》
中文版, 机械工业出版社

(4) Bjarne Stroustrup: 《C++程序设计语言 (特别版)》 中文版,
机械工业出版社





➤ 教学学时

课堂讲授24学时，上课12次，初步订在4、5、6月各停一次，第15周提前结束并笔试。停课我会提前一周通知。现在特殊时期，一切等大家返校后再说。

➤ 考核方式及成绩评定

平时作业占30%，期末笔试占40%，夏季小学期大作业占30%。本课没有机考。

➤ 作业与实验

平时作业上机完成，大约11次，从第1周开始，每次2-3个题，具体题目和要求参见相应课件。

➤ 夏季学期大作业（程序设计训练）

夏季小学期大作业以实验为主，不上课，两周时间，每人每天4个机时，共40个机时。完成2个大作业，并按照样例写实验报告。实验时间安排和具体要求到本学期末再通知。

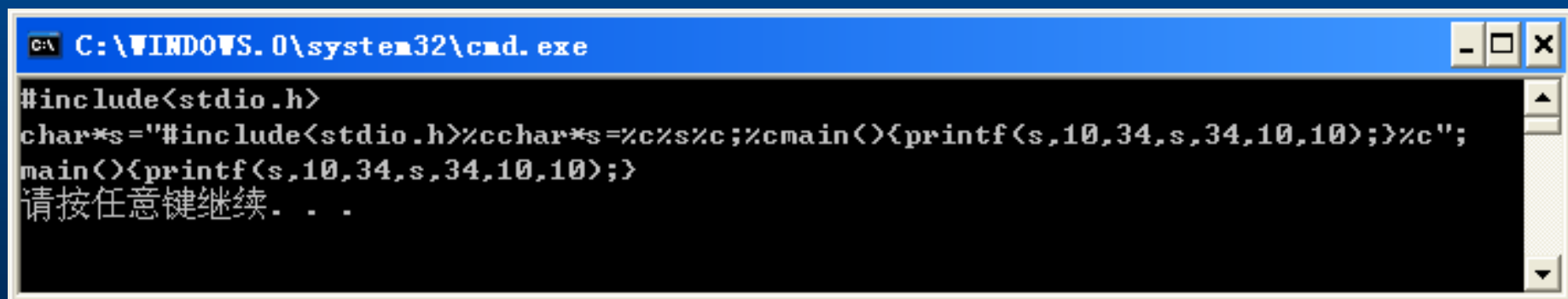
引言

有程序：

```
#include<stdio.h>
char*s="#include<stdio.h>%cchar*s=%c%s%c;%cmain(){printf(s,10,34,s,34,10,10);}%c";
main(){printf(s,10,34,s,34,10,10);}
```

运行的结果是：

```
#include<stdio.h>
char*s="#include<stdio.h>%cchar*s=%c%s%c;%cmain(){printf(s,10,34,s,34,10,10);}%c";
main(){printf(s,10,34,s,34,10,10);}
```



```
C:\WINDOWS.0\system32\cmd.exe
#include<stdio.h>
char*s="#include<stdio.h>%cchar*s=%c%s%c;%cmain(){printf(s,10,34,s,34,10,10);}%c";
main(){printf(s,10,34,s,34,10,10);}
请按任意键继续. . .
```

程序输出的结果和程序的源代码完全一样！

有程序：

```
main(){char *a;printf(a,34,a="main(){char *a;printf(a,34,a=%c%s%c,34);}",34);}
```

因为程序开头没有 **#include <stdio.h>**，此程序编译时有警告错误信息，但可以正常运行，运行的结果是：

```
main(){char *a;printf(a,34,a="main(){char *a;printf(a,34,a=%c%s%c,34);}",34);}
```

程序运行的结果和程序的源代码也完全一样！

C语言功能强大，希望大家进一步巩固所学过的C语言内容，这是下一步学习C++的基础。

注意：

C程序文件名后缀： **.c**

C++程序文件名后缀： **.cpp**

说明一下为什么刚才第二个程序输出是程序本身：

```
main()
```

```
{char *a;
```

```
    printf(a,34,a="main(char*a){printf(a,34,a=%c%s%c,34);}",34);  
}
```

34是字符“的ASCII值，函数参数传递顺序自右至左，因此输出语句：

```
printf(a,34,a="main(char*a){printf(a,34,a=%c%s%c,34);}",34);
```

等价于：

```
printf("main(){char*a;printf(a,34,a=%c%s%c,34);}",  
      34, "main(){char*a;printf(a,34,a=%c%s%c,34);}", 34);
```

第1个%c输出字符“， %s输出后面双引号中的字符串：

```
main(){char*a;printf(a,34,a=%c%s%c,34);}
```

第2个%c输出字符“

因此整个printf函数语句输出的是：

```
main(){char*a;printf(a,34,a="main(){char*a;printf(a,34,a=%c%s%c,34);}",34);}
```

如果你看到如下的源程序代码：

```
x = a.Get(a);  
a.Put(&a, x+1);
```

你认为这是C程序还是C++程序？

实际上这是C程序的一部分！

```
#include <stdio.h>
typedef struct A {
    int z;
    int (*Get)( ); /* 函数指针*/
    int (*Put)( ); /* 函数指针*/
} A;
int get(A a)
{ return a.z;
}
int put(A *a, int z)
{ a->z=z;
  return a->z;
}
void main( )
{ A a={2, get, put}; int x;
  x= a.Get(a);
  a.Put(&a, x+1);
  printf("%d\n", a.Get(a));
}
```

补充例题1 【字符串处理 去除C代码中的注释】

C/C++代码中有两种注释，`/* */`和`//`。编译器编译预处理时会先移除注释。就是把`/*`和`*/`之间的部分去掉，把`//`以及之后的部分删掉。这里约定，如果出现了`/* AAAA /* BBBB */`的情况，也就是`/* */`中出现了`/*`，那么第二个`/*`是不当作注释起始的。

编写函数 `void removeComment(char *str)`

分析：对于字符串

`"int c=4,/*c累计量*/ a=3;/*变量*/ // a初值为3 "`

先用`strstr`函数在`str`中确认`"/"`是否出现过，是则再确认`"*/"`是否出现过，是则把`str`中自`"*/"`出现位置后2个字符起始的字符串复制到`str`中`"/"`开始的位置，覆盖掉注释部分。循环查找直到找不到`"/"`为止；再用`strstr`在`str`中确认`"//"`是否出现过，是则把出现`"//"`的位置上置为`'\0'`。

```

#include <stdio.h>
#include <string.h>
void removeComment(char *str)
{
    char *p=str, *q;
    while ((p=strstr(p, "/*")) != NULL)
    {
        q=strstr(p, "*/");
        if (q != NULL)
            strcpy(p, q+2);
    }
    p = strstr(str, "//");
    if (p !=NULL)
        *p = '\0';
}
void main( )
{
    char s[]="int c=4, /*c累计量*/ a=3; /*变量*/ // a初值为3";
    removeComment(s);
    printf("%s\n", s);
}

```

运行结果: int c=4, a=3;

补充例题2【字符串处理 去除字符串中非字母字符】

编写函数 `void FilterNonChar (char *str)`

对于字符串

```
str= "int c=4,/*c累计量*/ a=3;/*变量*/ // a为3 "
```

最后结果应该是：

```
str= "intccaa"
```

分析：

设循环变量k从0到`strlen(str)`循环，逐个字符判断`str[k]`是否为字符，是留下，向前移动位置，不是忽略。

```
for(k=0; k<strlen(str); k++)
```

```
    if (str[k]>='a'&&str[k]<='z'
```

```
        || str[k]>='A'&&str[k]<='Z')
```

```
        // if (isalpha(str[k]))    // 此函数隶属于 ctype.h
```



```

#include <stdio.h>
#include <string.h>
#include <ctype.h>
void FilterNonAlpha(char *str)
{ int k, n=0;
  for(k=0; k<strlen(str); k++)
    if (str[k]>='a'&&str[k]<='z' || str[k]>='A'&&str[k]<='Z')
      // if (isalpha(str[k]))
        str[n++]=str[k];
  str[n] = '\0'; /* 设置新的字符串尾
}
void main( )
{ char s[]="int c=4,/*c累计量*/ a=3;/*变量*/ // a初值为3";
  FilterNonAlpha(s);
  printf("%s\n", s);
}

```

运行结果:

intccaa

请按任意键继续. . .

补充例题3【字符串处理 翻译机】

把一个句子中的单词的顺序倒过来。

输入：

theatre the to went I week Last

输出：

Last week I went to the theatre

分析：

如果能循环找出句子中的每一个空格，置为\0,设置一个指针数组，每一个指针指向每一个单词的开始处，逆向打印指针数组中的每一个单词，即可实现翻译机“把一个句子中的单词的顺序倒过来”的要求。

```
#include <stdio.h>
void main()
{ char s[1000], *p[100];
  int k=0,n=0;
  gets(s);
  p[0] = s;
  for(;s[k]; k++)
    if (s[k]==' ')
      { s[k]='\0'; n++;
        p[n]=&s[k+1];
      }
  for(k=n; k>=0; k--) // 逆序打印
    printf("%s ", p[k]);
}
```

运行结果:

theatre the to went I week Last
Last week I went to the theatre 请按任意键继续...

补充例题4【字符串处理 句子单词匹配】

从键盘上输入两个英文句子，输出他们相同单词的个数（句子最少包含一个单词，不包含标点符号，所有字母均为小写，单词与单词之间用'&'符号隔开）。

例如：输入

i&am&a&student
today&is&monday

输出 0

输入

now&is&to&two
to&classes

输出 1

分析：

如果能循环找出句子中的每一个&，置为\0，设置一个指针数组，每一个指针指向每一个单词的开始处，那么句子单词匹配问题就变成了循环比较两个指针数组中单词是否相同并计数。

```

#include <stdio.h>
#include <string.h>
void main()
{ char s1[1000],s2[1000],*p1[100],*p2[100];
  int k,m,n1=0,n2=0, same=0;
  gets(s1); gets(s2);
  p1[0] = s1; p2[0] = s2;
  for(k=0;s1[k]; k++)
    if (s1[k]=='&')
      { s1[k]='\0'; n1++; p1[n1]=&s1[k+1]; }
  for(k=0;s2[k]; k++)
    if (s2[k]=='&')
      { s2[k]='\0'; n2++; p2[n2]=&s2[k+1]; }
  for(k=0; k<=n1; k++)
    for(m=0; m<=n2; m++)
      if (!strcmp(p1[k],p2[m])) same++;
  printf("%d\n", same);
}

```

运行结果:

now&is&to&two

to&classes

1

请按任意键继续. . .

运行结果:

i&am&a&student

today&is&monday

0

请按任意键继续. . .

第1次作业:

见网络学堂第1次作业
2题必做, 1题选做

1. 编写程序从某个文本文件中读入若干个字符串（每个字符串长度不超过80个字符），将字符串按字典序（从小到大）排序，结果输出到另一个文本文件中。要求此程序能处理任意多个字符串。
2. 编写程序从一个二进制文件中读入若干个字节，将每一个字节的8位逆转后，按字节的输入顺序输出到另一个二进制文件中。这里的逆转是指：当unsigned char k=0xAC(二进制值为：10101100)，逆转后k为：0x35 (二进制值为：00110101)。并考虑当文件中的字节数量很多时（比如，几百MB, 几百GB甚至几百TB），如何优化此程序，使其执行效率最高。

3*.(探究题, 选做) 编写程序, 将一段文字中出现的所有英文阿拉伯数字和中文阿拉伯数字转化成(按照人们习惯的读音) 中文。比如: "我去百货大楼花2011元买了一块手表", 其中的: "2011"是英文阿拉伯数字, 每个数字占一个字节, 应该按照读音转化为: "两千零一十一"。也可能写作: "我去百货大楼花2 0 1 1元买了一块手表", 其中的"2 0 1 1"是中文格式的阿拉伯数字, 每个数字占两个字节, 也应该按照读音转化为: "两千零一十一"。又比如: "他花108050元买了一辆新速腾车", 其中的: "108050"应该按照人们读音习惯转化为: "十万八千零五十"。转化单位包括: 亿、万、千、百、十.....
更进一步, 可以转化带小数的数字, 例如将: "121.14156"转化为: "一百二十一点一四一五六"。

如何判断是否是英文阿拉伯数字和中文阿拉伯数字：

```
for(k=0; k<strlen(str); k++)  
    if (str[k]>='0' && str[k]<='9')
```

```
unsigned char zero[]=" 0 ", nine[]=" 9 ";  
for(k=0; k<strlen(str); k++)  
    if ((str[k]>=zero[0] && str[k]<=nine[0]) &&  
        (str[k+1]>=zero[1] && str[k+1]<=nine[1]))
```

注意： 不要出现跨字的问题，因为每个汉字占两个字节，顺序向后遍历字符串时，容易出现把第一个汉字的第二个字节与第二个汉字的第一个字节当做一个汉字来判断，出现错误。

C++程序设计

C++及创始人简介

- 1979年，刚从英国剑桥大学（CU）获得博士学位的29岁丹麦人Bjarne Stroustrup(本贾尼·斯特劳斯特卢普)。进入美国AT&T公司Bell(新泽西)实验室，开始在C语言的基础上研制C++，1983年研制出了C++的雏形。
- 最初1980年被称为：带类的C（C with Class），1983年正式命名为C++。从1989年开始C++的标准化工作，1994年推出了ANSI C++标准。
- 1998年11月ISO批准为ISO C++ 标准: C++98 (ISO/IEC 14882)
- 2011年9月ISO批准的最新C++标准是: C++11 (ISO/IEC 14882:2011)
- Bjarne Stroustrup 预计今年会产生: C++20

C++及创始人简介(2)

- Bjarne Stroustrup 在AT&T 的Bell实验室解散后，一度担任美国Texas A&M大学CS系的特聘教授。
- 近几年他担任了摩根士坦利公司技术部的Technical Fellow和Managing Director，兼任哥伦比亚大学CS系的访问教授。
- 他用自己的名字注册了网络域名，个人主页：

<http://www.stroustrup.com/>

- 他的个人感言：

I have stuck with its ISO standards effort for almost 30 years (so far).



Bjarne Stroustrup

Bjarne Stroustrup's Homepa X

www.stroustrup.com

火狐官方网站 新手上路 常用网址 京东商城

移动设备上的书签

Welcome to Bjarne Stroustrup's homepage!



I'm a Technical Fellow and a Managing Director in the technology division of [Morgan Stanley](#) in New York City and a Visiting Professor in Computer Science at [Columbia University](#).

I designed and implemented [the C++ programming language](#). To make C++ a stable and up-to-date base for real-world software development, I have stuck with its ISO standards effort for almost 30 years (so far).

- [A Tour of C++ \(2nd edition\)](#) (a brief - 240 page - tour of the C++ Programming language and its standard library for

Stroustrup: C++

http://www.stroustrup.com/C++.html

The C++ Programming Language

Modified April 28, 2019

C++ is a general-purpose programming language with a bias towards systems programming that

- is [a better C](#)
- supports [data abstraction](#)
- supports [object-oriented programming](#)
- supports [generic programming](#).

Or, in other words: C++ is a language for defining and using light-weight abstractions. It has significant strengths in areas where hardware must be handled effectively and there are significant complexity to cope with. This includes many resource constrained systems and much foundational and infrastructure code.

Translations:

- [Belorussian](#).
- [Spanish](#).
- [Russian](#).

I ([Bjarne Stroustrup](#)) am the designer and original implementor of C++. You can find the language, the techniques for using it, and the techniques for implementing it described in my [books](#), my [papers](#), in hundreds of books by others, and thousands of papers by others. There are far too many to list. Try a bookstore or a library. Answers to many questions about C++ can be found in

- my [FAQ](#),
- my [C++ Style and Technique FAQ](#)
- my [C++ glossary](#), and

完成

热点推荐

Stroustrup: C++

http://www.stroustrup.com/C++.html

- The ISO C++ Standard: C++ is standardized by ISO (The International Standards Organization) in collaboration with national standards organizations, such as ANSI (The American National Standards Institute), BSI (The British Standards Institute), DIN (The German national standards organization). The original C++ standard was issued in 1998, a minor revision in 2003, and a major update, C++11, was issued in September 2011, and the current standard is C++14. During its development, C++11 was referred to as C++0x. After that, C++14 and C++17 were delivered according to a new ambitious 3-year schedule. Currently the standards committee is working to produce a new standard, a major revision, in 2020: C++20.
 - The of [The C++ Foundation's](#) site for information about [ISO C++ standards activities](#). Updated regularly.
 - [An almost complete C++14 standard](#). Note that this is most certainly not a tutorial. You can get the official final version from the ISO or NIST for \$60. You are unlikely to need that unless you are a compiler implementer.
 - [The committee's current working paper](#).
 - The ISO C++ standards committee (WG21) maintains an [official site](#) with information about the current state of the standards effort. "More than you ever wanted to know about the work on the C++ standard."
 - [My view of what C++17 should be](#). April 2015. Note that I don't always get what I want and that I'm aggressive about the improvement of C++.
 - My book [The Design and Evolution of C++](#) describes the standards process and many of the design decisions made
 - My book [The C++ Programming Language \(Fourth Edition\)](#) describes C++ as defined by the ISO standard.
- How to write good modern C++: Much C++ code is written in archaic styles, missing out on elegance, safety and performance. This is avoidable.
 - [A paper of how to write guaranteed type and resource safe C++](#).
 - [A set of guidelines for writing good, modern, efficient C++](#) on [Github](#).
 - [A Tour of C++ \(second edition\)](#): a short book (240 pages) providing an overview of C++ as it is in 2015. Aimed at people who can program, but might have a 1990s view of C++.
- Applications, compilers, etc.:
 - A list of [interesting C++ applications](#). I welcome suggestions for additions.
 - [A list of major industry applications and tools](#) with evolution paths by Vincent Lextrait.
 - An incomplete [list of C++ compilers](#).
 - Hans-J. Boehm's [site for C and C++ garbage collection](#) and a couple of sites offering collectors based on his work

完成 热点推荐 100%

C++及创始人简介(3)

C++被称为面向对象的程序设计语言

OOPL (Object-Oriented Programming Language)

C是C++的子集，C++保持了C的原始思想，程序员写程序时可以用也可以不用C++所增加的功能。

C++在C的关键字的基础上增加了：catch, class, const, delete, friend, inline, new, operator, private, protected, public, template, this, throw, try, typeid, virtual, volatile 等关键字。

C++在C语言中增加的最重要的机制有三个：

- 类 (class)
- 函数重载 (function overloading)
- 操作符重载 (operator overloading)

C++及创始人简介(4)

主要目的:

1. 用于大型软件(指源程序超过5万行)的工业化软件开发。
2. 解决C语言程序访问权限不受任何限制而导致的副作用严重的问题:
 - a. 全局变量全局可见可任意访问修改
 - b. 不检查数组越界引起的致命错误
3. 生成严格控制存取权限的“黑盒子功能单元”
---对象 (object)

1. 关于C++的几个基本问题

1.1 基本数据类型

1.2 注释

1.3 输入/输出

1.4 内联函数 (inline)

1.5 动态内存分配

1.6 指针/地址的传递

1.7 类型修饰符const

1.8 作用域与可见性

1.9 缺省参数

1.10 两个基本概念

1.1 基本数据类型

C++全面继承了C的基本数据类型: char,short,int,long, float,double,long long等，并引入了新的类型：

bool（布尔量） 取值只有两个：true/false

给bool赋值别的数，都将强制转换成true（不为0时）或false（为0时）。bool型占1个字节，等同于char。

```
#include <stdio.h>
```

```
void main( )
```

```
{ bool f1, f2; int i;
```

```
  f1=true; f2=false;
```

```
  printf("%d %d\n", f1, f2); // 结果: 1 0
```

```
  i = (f1+1)* 10;
```

```
  printf("%d %d\n", sizeof(f1), i); // 结果: 1 20
```

```
}
```


1.2 注释

C语言用 `/*` 和 `*/` 进行注释，从 `/*` 开始一直到 `*/` 之间为注释信息。

C++程序中增加了**行注释**：`//`，从 `//` 开始到本行尾（回车）为注释信息。

例如：

```
void main()  
{ int i;      // i是循环变量  
  float sum;  // sum用来求累加和  
  .....  
}
```

1.3 输入/输出

C++利用cin和cout这两个流类对象进行输入/输出，完全是自由格式。例如：

```
cin >> i >> f; // 等同于 scanf("%d%f", &i, &f);
```

```
cout << i << f << endl;
```

```
// 等同于 printf("%d%f\n", i, f);
```

```
// 其中 endl 等同于 '\n'
```

源程序开头的文件引用改为：

```
#include <iostream.h>
```

而不是C源程序所文件引用的：

```
#include <stdio.h>
```

源程序开头的文件引用：

```
#include <iostream.h>
```

也可以写成：

```
#include <iostream>
```

```
using namespace std;
```

这是目前ISO C++建议的写法，意思是：使用标准命名空间，这样#include所引用的头文件将是不带.h的。

原来C语言的stdio.h, string.h, math.h等文件引用仍可使用,但需要去掉文件名后缀.h，前面加上c：

```
#include <cstring>
```

```
#include <cmath>
```

对于一般的输入输出操作上面两种写法没有本质区别，但文件操作将会有较大差别。

ISO C++ 写法:

```
#include <iostream>
using namespace std;
void main(void)
{ int i=33, j=35, k=12345;
  float f=3.14159f;
  cout << i << j << endl;
  cout << k;
  cout << i << j << endl;
  cout << j << f << endl;
}
```

输出结果:

3335

123453335

353.14159

也可以写为：（这个写法最标准）

```
#include <iostream>
using namespace std;
int main( )
{ int i=33, j=35, k=12345;
  float f=3.14159f;
  cout << i << j << endl;
  cout << k;
  cout << i << j << endl;
  cout << j << f << endl;
  return 0; // 主函数的返回值，这个返回值任意
}
```

输出结果：

3335

123453335

353.14159

1.3 输入/输出(续1)

常用的I/O流类库操纵符(函数):

`setw(int)` 设置随后(一个)输出内容的场宽,不够自动突破

`setprecision(int)` 设置随后输出的(一个)浮点数的位数(不包括小数点)

`hex` 随后所有的整型数值采用十六进制输出

`oct` 随后所有的整数值采用八进制输出

`dec` 随后所有的整数值采用十进制输出

`endl` 回车符

要想使用`setw`和`setprecision`, 还必须引用 `iomanip.h`

程序开始处加: `#include <iomanip.h>`

或 `#include <iomanip>`

```

#include <iostream>
#include <iomanip>
using namespace std;
void main(void)
{
    int i=33, j=35, k=12345;
    float f=3.14159f;
    cout << setw(10) << i << setw(8) << j << endl;
    cout << setw(3) << k << endl;           // 场宽不足,自动突破
    cout << hex << i << setw(3) << j << endl; // 用十六进制输出整数
    cout << i << setw(3) << j << endl;
    cout << oct << i << setw(j/8) << j << endl; // 用八进制输出整数
    cout << setw(12) << setprecision(5) << f << endl;
    cout << setw(4) << setprecision(6) << f << endl; // 场宽不足,自动突破
    cout << setprecision(4) << f << endl;
                                   // 未定场宽,按照实际宽度输出
    cout << setprecision(1) << f << endl;
    cout << setprecision(2) << 314.159 << endl;
}

```

运行结果:

33 35

12345

21 23

21 23

41 43

3.1416

3.14159

3.142

3

3.1e+002

1.3 输入/输出 字符串

```
char s1[20], s2[20], s3[20];
```

```
cin >> s1 >> s2 >> s3;
```

```
// 等同于 scanf("%s%s%s", s1, s2, s3);
```

```
cout << s1 << s2 << s3 << endl;
```

```
// 若从键盘上输入: How are you?
```

```
// 输出结果是: Howareyou?
```

整行输入字符串:

```
cin.getline(s1, N, 结束符); // '\n' 为默认结束符
```

```
cin.get(s2, N, 结束符); // 一次连续读入多个(包括空格)
```

```
// 字符, 直到读满N-1个, 或遇到 结束符
```

```
// getline读取但不存结束符, get既不读取也不存结束符
```

```
#include <iostream>
using namespace std;
void main( )
{ char s1[20], s2[20], s3[20];
  cin.getline(s1, 20, '\n'); // 最多读入19个字符
  cin.get(s2, 20, '\n');
  cin.get(s3, 20, '\n');
  cout << s1 << s2 << s3;
}
```

运行结果:

abcd

efgh

abcdefgh请按任意键继续...

可以看到get读s2时未读掉回车符，导致读s3又遇到回车符，未读入内容，s3为空字符串。

```
#include <iostream>
using namespace std;
void main( )
{ char s1[20], s2[20], s3[20];
  cin.getline(s1, 20, '\n'); // 最多读入19个字符
  cin.getline(s2, 20, '\n');
  cin.getline(s3, 20, '\n');
  cout << s1<<"==" << s2 <<"=="<< s3<<"==";
}
```

运行结果:

123456789012345678901234567890

1234567890123456789=====请按任意键继续...

可以看到getline读入了19个字符给s1，但并没有把这一行剩余的字符读入给s2，s3也未读入字符，为什么？？？

因为getline读入19个字符给s1时没有遇到结束符，因而产生错误，cin处于错误状态，需要清除错误状态后才能继续读入。


```
#include <iostream>
using namespace std;
void main( )
{ char s1[20], s2[20], s3[20];
  cin.getline(s1, 20, '\n'); // 最多读入19个字符
  cin.clear( ); //清除cin输入对象的错误状态
  cin.getline(s2, 20, '\n');
  cin.getline(s3, 20, '\n');
  cout << s1<<"==" << s2 <<"=="<< s3<<"==";
}
```

运行结果：

123456789012345678901234567890

123456789012345

1234567890123456789==01234567890==123456789012345==请
按任意键继续...

可以看到getline读入了19个字符给s1，又把同一行剩余的字符直到'\n'读入给s2，并继续等待输入，又读入字符串给s3。

```
#include <iostream>
using namespace std;
void main( )
{ char s1[20], s2[20], s3[20];
  cin.getline(s1, 20, '\n'); // 最多读入19个字符
  cin.clear( ); //清除cin输入对象的错误状态
  cin.get(s2, 20, '\n'); // getline 改为get
  cin.getline(s3, 20, '\n');
  cout << s1<<"==" << s2 <<"=="<< s3<<"==";
}
```

运行结果:

123456789012345678901234567890

1234567890123456789==01234567890====请按任意键继续...

可以看到把第二个getline改为get后，getline读入了19个字符给s1，get把同一行剩余的字符直到'\n'读入给s2，但并没有读掉'\n'，getline读入空字符串给s3。

1.4 内联函数 (inline)

为提高执行效率，C++增加了内联函数。

与宏定义define类似，编译器将出现函数名的位置用函数体代码进行替换，即所谓在线化，这样省去了函数调用的参数入栈、退栈、跳转等所花费的时间。

内联函数比define方便，功能也更强，不只是简单的**字符串的替换**，全面考虑了**类型匹配**问题。

例如：

```
#include <iostream>
using namespace std; // ISO C++写法
inline double Area(double R) // 根据半径R求圆面积
{ return 3.14159*R*R; }
void main( )
{ double r(3.0); // 赋初值，等同于 double r=3.0;
  double area;
  area = Area(r); // 执行效率等同于 area=3.14159*r*r;
  cout << area << '\n';
}
```

运行结果： 28.2743

1.4 内联函数（inline）(续)

内联函数的书写有一些限制(但没有统一的明确规定，由各家的编译器的聪明程度决定)：

- 内联函数中可以定义变量，可以使用循环语句、分支语句等；
- 但内联函数不能直接递归或间接递归；
- 其它使编译器不能把函数调用在线化的情形。

凡是编译器没法把函数调用在线化的内联函数，编译器都会自动忽略inline，把它当做一般函数来调用，inline也就自动失效了。

```
#include <iostream>
using namespace std;
inline double Area(double R)
{   if (R>0)
        return Area(R-1) + 3.14159*R*R;
    return 3.14159*R*R;
}
void main( )
{   double r(3.0); double area;
    area = Area(r);
    cout << area << '\n';
}
```

如果把前面例子的Area函数改为自递归函数，照样可以编译运行，运行结果： 43.9823

但实际上Area并没有被当做内联函数。

1.5 动态内存分配

C++用new和delete动态分配和释放内存，等同于C的malloc和free函数，但new和delete功能更强大。

例如： `int *p=new int(10);`

`//分配1个 int 动态内存单元,并赋初值为10`

等同于C的：

`int *p=(int *)malloc(sizeof(int));`

`*p = 10;`

C++用：

`delete p; // 释放p所指的1个int单元`

等同于C的：

`free(p);`

1.5 动态内存分配(续1)

`int *pt=new int[10];` // 分配10个int单元,
// 即pt是一个动态数组, `pt[0]~pt[9]`
等同于C的:

`int *pt = (int *)malloc(sizeof(int)*10);`

C++用:

`delete []pt;`

释放pt所指的10个连续int单元, 等同于C的:

`free(pt);`

提醒: 注意 `pt=new int(10);` 与 `pt=new int[10];` 的差别

1.5 动态内存分配(续2)

```
int (*pt)[10]=new int[8][10]; //分配80个int单元
```

// pt是一个二维动态数组, pt[0][0]~pt[7][9]

等同于C的:

```
int (*pt)[10];
```

```
pt = (int (*)[10])malloc(sizeof(int)*8*10);
```

C++用:

```
delete []pt;
```

释放行指针pt所指的80个连续int动态内存单元,

等同于C的:

```
free(pt);
```

1.5 动态内存分配(续3)

```
int **q = new int *[10]; //分配10个int指针单元  
// q是一个一维动态指针数组, q[0]~q[9]
```

等同于C的:

```
int **q;  
q = (int **)malloc(sizeof(int *)*10);
```

C++用:

```
delete []q;
```

释放q所指的10个连续int指针单元, 等同于C的:

```
free(q);
```

例如:

```
#include <iostream>
using namespace std;
void main( )
{  int **q = new int *[10];
   int a[10]={1,2,3,4,5,6,7,8,9,10}, k;
   for (k=0; k<10; k++)
       q[k] = &a[k];
   for (k=9; k>=0; k--)
       cout << *q[k] << ',';
   delete [ ]q;
}
```

运行结果:

10,9,8,7,6,5,4,3,2,1,请按任意键继续...

1.6 指针/地址的传递

- ◆ C的参数传递只是**传值**,无法将变量地址传递到函数中。除非通过显式的指针方式传递。
- ◆ C++提供了引用机制来**传递变量地址**。

方法是:

- ◆ 在函数头部要传递地址的形参名前加**&**，而函数调用方式不变。
- ◆ 这样在被调用函数中对声明**传递变量地址**的变量的赋值操作，实际上是直接对调用函数中相应变量的操作。

例如：交换两个变量的值(C版本)

```
#include <iostream>
```

```
using namespace std;
```

```
void swap(int a, int b) // 将a, b变量的值交换
```

```
{ int t=a;
```

```
    a=b; b=t;
```

```
}
```

```
void main( )
```

```
{ int i=3, j=5;
```

```
    swap(i, j);    //交换i, j的值
```

```
    cout << i << "    " << j << endl ;
```

```
}
```

输出结果将是： 3 5 a和b的值并未交换

在C语言方式下除非改成传递地址的指针方式:

```
#include <iostream>
```

```
using namespace std;
```

```
void swap(int *a, int *b) // 将a,b所指变量的值交换
```

```
{ int t=*a;
```

```
    *a = *b; *b=t;
```

```
}
```

```
void main( )
```

```
{ int i=3, j=5;
```

```
    swap(&i, &j); //交换i, j的值
```

```
    cout << i << "    " << j << endl ;
```

```
}
```

输出结果将是: 5 3 // a和b的值已经交换

C++引用声明传地址方式:

```
#include <iostream>
```

```
using namespace std;
```

```
void swap(int &a, int &b) // 引用声明传递a,b变量的地址
```

```
{ int t=a;
```

```
    a=b; b=t;
```

```
}
```

```
void main( )
```

```
{ int i=3, j=5;
```

```
    swap(i, j);    //交换i, j的值
```

```
    cout << i << "    " << j << endl ;
```

```
}
```

输出结果将是: 5 3 // a和b的值已经交换

又例如：交换两个指针变量的值

```
#include <iostream>
```

```
using namespace std;
```

```
void swap(int *a, int *b) // 将a, b指针变量的值交换
```

```
{ int *t;
```

```
    t=a; a=b; b=t;
```

```
}
```

```
void main( )
```

```
{ int i=3, j=5, *p=&i, *q=&j; // p指向i, q指向j
```

```
    swap(p, q); //交换p, q指针的值, 让p指向j, q指向i
```

```
    cout << *p << "    " << *q << endl ;
```

```
}
```

输出结果将是： 3 5 // p和q的值并未交换

除非传递指针变量的地址:

```
#include <iostream>
```

```
using namespace std;
```

```
void swap(int **a, int **b) // 将a,b所指变量的值交换
```

```
{ int *t;
```

```
    t=*a;    *a=*b;    *b=t;
```

```
}
```

```
void main( )
```

```
{ int i=3, j=5, *p=&i, *q=&j; // p指向i, q指向j
```

```
    swap(&p, &q); //交换p,q指针的值,让p指向j,q指向i
```

```
    cout << *p << "    " << *q << endl ;
```

```
}
```

输出结果将是: 5 3 // p和q的值已交换

C++引用声明传指针变量地址方式:

```
#include <iostream>
```

```
using namespace std;
```

```
void swap(int *&a,int *&b)// 引用声明传递指针a,b变量的地址
```

```
{ int *t;
```

```
    t=a; a=b; b=t;
```

```
}
```

```
void main( )
```

```
{ int i=3, j=5, *p=&i, *q=&j; // p指向i, q指向j
```

```
    swap(p, q); //交换p, q指针的值, 让p指向j, q指向i
```

```
    cout << *p << "    " << *q << endl ;
```

```
}
```

输出结果将是: 5 3 p和q的值已经交换, p指向了j,
q指向了i, 但i=3,j=5不变

- ✓引用方式（传变量地址）除了能简化程序书写，还能提高程序的执行效率，因为当传递的是结构体(对象)时，只需传递其地址，让函数直接访问，而不必按照C语言传值的方式将结构体(对象)复制到函数的形参表中。
- ✓后面会看到：对于类的复制构造函数来说，这种引用方式（传变量地址）的机制，不是可有可无，而是必须的。
- ✓函数返回值也可以返回引用，这样也只需传递所返回量的地址，让调用函数直接访问，而不必按照C语言传值的方式将返回量复制后传递返回。但**不能返回局部变量的引用**。

例如, 下例max函数返回的是引用:

```
#include <iostream>
using namespace std;
const int &max(int &a, const int &b)
{   return a>b? a:b;
    // 返回的是a或b也就是i或j的地址
}
void main( )
{   int i=3, j=5, k;
    k=max(i, j); //取i, j的值
    cout << k << endl ;
}
```

1.6 指针/地址的传递(续1)

引用方式:

引用(reference)是一种特殊类型的变量,可以认为是给一个变量起别名。

```
int i, j;
```

```
int &r=i; // 变量引用, r和i是同一个内存单元
```

// r不能再改变引用,可以认为r是i的别名

```
int *p=&i; // 注意指针与引用的区别
```

```
i=10;
```

```
j=r;           // 等价于 j=i
```

```
*p = 20;       // 等价于 i=20
```

1.6 指针/地址的传递(续2)

```
#include <iostream>
using namespace std;
void main( )
{ int i=10, j=20, k=30, *p=&i;
  int *&q=p; //指针引用, q和p是同一个指针单元
  // q不能再改变引用, 可以认为q是p的别名
  cout << *p << ' ' << *q << endl;
  p = &j;
  cout << *p << ' ' << *q << endl;
  q = &k;
  cout << *p << ' ' << *q << endl;
}
```

运行结果为:

10 10

20 20

30 30

1.7 类型修饰符const

- ◆ `const`是类型修饰符，在所定义的变量类型前面或后面出现，使之成为不可修改的量，常量。

例如，`int const a=1;` 或 `const int a=1;`
使变量a成为不可修改的量，等同于C的常量，但
`const`比C语言的常量功能更强,因为`const`是带类型的。

- ◆ `const`经常用于函数参数说明，防止在函数中无意破坏参数的值或所指内容。

例如，`strcpy(char *str2, const char *str1);`
是将字符串str1复制到str2。将参数str1定义为`const`，
就是防止在`strcpy`函数中破坏str1所指的字符串。

- ◆ 尤其是在C++引入传地址的引用机制后，`const`用来保证在函数中不会修改相应参数变量的值或所指内容。

例如：

```
#include <iostream>
```

```
using namespace std;
```

```
void swap(int &a, const int &b) // b是常引用
```

```
{ int t=a;
```

```
    a=b;
```

```
    b=t;           // 第6行
```

```
}
```

```
void main( )
```

```
{ int i=3, j=5;
```

```
    swap(i, j);    //交换i, j的值
```

```
    cout << i << " " << j << endl;
```

```
}
```

编译结果：

```
swap_const.cpp(6) : error C3892: “b”: 不能给常量赋值
```


又例如：

```
#include <iostream>
using namespace std;
void swap(int &a, const int &b) // b是常引用
{
    int t=a; a=b;
    b=t; // 第5行
}
void main( )
{
    int const i=3, j=5; // i和j是整型常量
    i=4; // 第9行
    j=6; // 第10行
    swap(i, j); //交换i, j的值
    cout << i << " " << j << endl ;
}
```

编译结果：

test.cpp(5) : error C3892: “b”: 不能给常量赋值

test.cpp(9) : error C3892: “i”: 不能给常量赋值

test.cpp(10) : error C3892: “j”: 不能给常量赋值

1.7.1 共用数据的保护：常对象

例如：

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    int  sno;
};
const STU a={"Zhang San", 'm', 2019000001}; // 常对象
// 也可以写为： STU const a={"Zhang San", 'm', 2019000001};
void main() // C++结构体类型定义变量不必再写struct
{ STU b={"Li Shuang", 'f', 2019000002};
  b=a;
  cout<<b.name<<" " << b.gender <<" " << b.sno <<" " <<endl ;
}
编译没有错误，运行结果：
Zhang San m 2019000001
```

如果将语句: `b=a;`
改为: `a=b;`

编译结果:

test.cpp(11) : error C2678: 二进制 “=”: 没有找到接受 “const STU” 类型的左操作数的运算符(或没有可接受的转换)

1> test.cpp(7): 可能是 “STU &STU::operator =(const STU &)”

1> 试图匹配参数列表 “(const STU, STU)”时

因为a是一个常量对象，简称为: 常对象

除了定义时一次性赋初值，不能再改变它的值，从而共用时起到保护作用。

1.7.2 共用数据的保护：常数据成员

例如：

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    static int const sno=1; // 静态常数据成员
};
STU const a={"Zhang San", 'm'}; // 常对象
void main( )
{ STU b={"Li Shuang", 'f'}; // 不能再给sno赋值
  b=a;
  cout<<b.name<<" "<<b.gender<<" "
    <<b.sno<<" "<<endl;
}
```

编译没有错误，运行结果：

Zhang San m 1

但是结构体中成员sno前面的static不能去掉，否则在VS2008编译器上会出现编译错误：

test.cpp(6) : error C2864: “STU::sno”: 只有静态常量整型数据成员才可以在类中初始化

test.cpp(8) : error C3852: “STU::sno”具有类型 “const int”: 聚合初始化未能初始化该成员

1> 常量成员不能被默认初始化，除非它们的类型具有用户定义的默认构造函数

test.cpp(8) : error C2512: “STU::STU”: 没有合适的默认构造函数可用

test.cpp(8) : error C2078: 初始值设定项太多

test.cpp(10) : error C3852: “STU::sno”具有类型 “const int”: 聚合初始化未能初始化该成员

1> 常量成员不能被默认初始化，除非它们的类型具有用户定义的默认构造函数

test.cpp(10) : error C2512: “STU::STU”: 没有合适的默认构造函数可用

test.cpp(10) : error C2078: 初始值设定项太多

test.cpp(11) : error C2582: “operator =”函数在 “STU”中不可用

编译器提示： 只有静态常量整型数据成员才可以在类中初始化。

1.7.3 共用数据的保护：指向对象的常指针

例如：

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    int  sno;
};
STU a={"Zhang San", 'm', 2019000001};
STU * const p = &a; // 常指针，此常指针p只能初始化时赋值
void main( )
{ STU b={"Li Shuang", 'f', 2019000002};
  p=&b; // 第12行
  cout<<p->name<<" "<< p->gender<<" "<<p->sno<<" "<<endl;
}
```

编译结果： test.cpp(12) : error C3892: “p”: 不能给常量赋值

1.7.4 共用数据的保护：指向常对象的指针

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    int sno;
};
const STU a={"Zhang San", 'm', 2019000001};
STU * const p = &a;          // 第9行 常指针不能指向常对象
void main( )
{ STU b={"Li Shuang", 'f', 2019000002},*q;
  q=&a;                      // 第12行 普通指针也不能指向常对象
  cout<<p->name<<" "<<p->gender<<" "<<p->sno <<" "<<endl;
}
```

编译结果：

test.cpp(9) : error C2440:"初始化": 无法从"const STU *"转换为"STU *const"
转换丢失限定符

test.cpp(12) : error C2440: “=”: 无法从 “const STU *”转换为 “STU *”
转换丢失限定符

改为:

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    int sno;
};
const STU a={"Zhang San", 'm', 2019000001};
const STU * p = &a;    // 第9行 p是指向常对象的指针
void main()
{ STU b={"Li Shuang", 'f', 2019000002};
  p=&b; // 第12行 p不是常指针, 可以被修改
  cout<<p->name<<" "<<p->gender<<" " <<p->sno <<" "<<endl;
}
```

编译没有任何错误, 运行结果:

Li Shuang f 2019000002

结论: 指向常对象的指针不是常指针, 可以被修改, 也可以指向普通对象。

1.7.5 共用数据的保护：指向常对象的常指针

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    int sno;
};
const STU a={"Zhang San", 'm', 2019000001}; // 常对象
const STU * const p = &a; // 第9行 p是指向常对象的常指针
void main( )
{ STU b={"Li Shuang", 'f', 2019000002};
  p=&b; // 第12行 p是常指针，不能被修改
  cout<<p->name<<" "<<p->gender<<" "<<p->sno <<" "<<endl;
}
```

编译结果：test.cpp(12) : error C3892: “p”: 不能给常量赋值

此外还有：常成员函数 void get() **const** { } 等讲到类时再解释。

1.8 作用域与可见性

- 在C语言中可以出现局部变量和全局变量同名，但函数中的局部变量将掩蔽全局变量（mask），直到局部变量的作用域失效，才能访问同名的全局变量。
- 而实际应用中可能需要同时访问同名的局部变量和全局变量。
- C++为了克服这个缺点，引入了作用域操作符(scope operator) ‘::’，使我们可以同时访问同名的局部变量(max)和全局变量(::max)。

例如:下例中max是局部变量, ::max是全局变量

```

#include <iostream>
using namespace std;
const int max=10000;
void main( ) // 从键盘上读入10个数，求最大值max输出,但最大不超过10000
{ int max, i,j;
  cin >> max;
  for (j=1; j<10;j++)
  {   cin >> i;
      if (i>max) max = i;
      if (max > ::max) max = ::max;
  }
  cout << max<<endl;
}

```

若max大于外部变量max的值，则取外部常量max的值，这种做法通常用来防止数组越界或溢出。

1.9 缺省参数

在函数声明中可以为一个或多个参数指定缺省值，这样函数调用时可以从右边开始省略一个或多个参数。例如，若有以下函数说明：

```
void delay(int day=1, int month=3, int year=2019)
{.....}
```

则以下函数调用都是正确的：

```
delay();      // 将使day=1, month=3, year=2019;
delay(29);    // 将使day=29, month=3, year=2019;
delay(29, 4); // 将使day=29, month=4, year=2019;
delay(27, 2, 2020);  // 将使day=27, month=2, year=2020;
```

为方便函数的使用，C++的库函数参数采用了缺省方式。比如：`cin.getline(s1, 20);` // 缺省 `'\n'` 为结束符

```
#include <iostream>
using namespace std;
void delay(int day=1, int month=3, int year=2019)
{ cout << month << '/' << day << '/' << year << endl;
}
void main( )
{ delay( );           // 将缺省使day=1, month=3, year=2019
  delay(29);          // 将使day=29, month=3, year=2019
  delay(29, 4);        // 将使day=29, month=4, year=2019
  delay(27,2,2020);    // 将使day=27, month=2, year=2020
}
```

运行结果:

3/1/2019

3/29/2019

4/29/2019

2/27/2020

请按任意键继续...

```
#include <iostream>
using namespace std;
void main( )
{ char s1[20], s2[20], s3[20];
  cin.getline(s1, 20); //省略了第3个输入结束符参数,缺省为 '\n'
  cin.get(s2, 20, '\n');
  cin.get(s3, 20); //省略了第3个输入结束符参数,缺省为 '\n'
  cout << s1 << s2 << s3;
}
```

运行结果:

abcd

efgh

abcdefgh请按任意键继续...

运行结果与原来的一样，可以看到getline和get都缺省了第3个输入结束符参数，自动缺省值为'\n'。

1.10 两个基本概念

数据抽象 (data abstraction)

- 将数据类型与操作分开，被称为数据抽象。
- 例如：栈操作，操作对象的类型可以是int,long,float,double等等，但不考虑类型，就可以成为一个数据抽象——栈操作，可以由此写出一个栈操作模板，适用于任何数据类型的栈操作。

例如： `T abs(T i)`

```
{ return i<0 ? -i : i; }
```

abs是一个函数模板 (function template)，可对任意类型的数据求绝对值，只要此数据可以和0比较大小。

对象 (object)

- 将一个(组)变量，以及作用于这些变量的一个(组)函数封装为一个单元，这个单元就被称为对象。类是同类对象的抽象(共同特性的描述)，对象是类的实例化。
- 对象具有封闭性，可以自己控制外界对它的访问权限，从而严格控制外界对其中被封装变量的访问。



第2次作业:

见网络学堂第2次作业

2题必做, 1题选做